

Manuale Tecnico (Estratto)

Titolo: Manuale Tecnico – Progetto The Knife **Versione:** 1.0 | **Data:** 06/07/2026

1. Architettura del sistema

Il progetto adotta un'architettura **Client-Server** distribuita, basata su **Java RMI** (Remote Method Invocation).

- **Client:** Gestisce l'interfaccia utente (JavaFX) e invia richieste al server.
- **Server:** Agisce come strato di astrazione tra il client e il database, orchestrando la logica di business.

2. Gestione Database (HikariCP)

Il sistema utilizza **HikariCP** per il *connection pooling*. Questa scelta algoritmica permette di:

- Ridurre l'overhead di apertura/chiusura connessioni.
- Gestire in modo resiliente le eccezioni di rete verso PostgreSQL.
- *Frammento di codice (Configurazione):*

Java

```
HikariConfig config = new HikariConfig();
```

```
config.setJdbcUrl("jdbc:postgresql://localhost:5432/TheKnifeVirgilio");
```

3. Struttura Dati e UML

- **Diagramma di Classe:** La struttura statica riflette il pattern DAO (*Data Access Object*), separando nettamente la persistenza dei dati dalla logica applicativa.
- **Diagramma di Sequenza:** Le interazioni tra client e server sono mediate da interfacce remote che definiscono i contratti di comunicazione RMI.

4. Design Patter

- **Data Access Object (DAO) Pattern**

Questo è il pattern cardine dell'intera architettura di persistenza.

Descrizione: Abbiamo separato la logica di business dalla logica di accesso ai dati (SQL/PostgreSQL).

Vantaggio: Il codice "di business" non conosce come i dati vengono letti o scritti (non vede le query SQL). Se in futuro dovessi decidere di cambiare database (es. passare da PostgreSQL a MySQL), dovrei modificare solo le classi DAO, lasciando intatto il resto del programma.

- **Singleton Pattern (Gestore Connessioni)**

Utilizzato per gestire la connessione al database.

Descrizione: Assicura che esista una sola istanza del gestore del pool di connessioni (HikariDataSource) per l'intera applicazione.

Vantaggio: Previene l'apertura indiscriminata di connessioni multiple che potrebbero saturare le risorse del database PostgreSQL, centralizzando il punto di accesso e garantendo una gestione efficiente delle risorse di rete.

- **MVC (Model-View-Controller) - Variante JavaFX**

Il progetto segue i principi del pattern MVC nella gestione dell'interfaccia grafica.

Model: Le classi che rappresentano i dati (es. Ristorante, Utente, Prenotazione).

View: I file .fxml e le relative classi (es. DettaglioRistoranteAnonimoView) che si occupano esclusivamente della presentazione.

Controller: Gestisce l'interazione tra l'utente e il Model, aggiornando la View in risposta ai comandi (es. il listener del mouse sulla lista dei risultati).

- **Observer Pattern (Listener delle View)**

Utilizzato per la gestione degli eventi dell'interfaccia utente.

Descrizione: Il sistema reagisce dinamicamente alle azioni dell'utente (es. setOnMouseClicked, setOnCloseRequest).

Vantaggio: Permette di disaccoppiare l'interfaccia dalla logica di controllo, rendendo il sistema reattivo e facile da testare.

5. Javadoc

Il codice sorgente è interamente documentato tramite standard Javadoc. Ogni classe di business e interfaccia remota include commenti descrittivi con tag @param, @return e @throws, conformemente a quanto indicato in **03_LabB_DocumentazioneProgetto.pdf**. La documentazione HTML generata è consultabile in `/target/site/apidocs/index.html`.